Методы обраьотки изображений

Лабораторная работа 3

- Выполнил: Кузьмин Артемий Андреевич
- Проверил: Потемин Игорь Станиславович

Цели работы

Изучить методы формирования заданных распределений случайных координат и направлений лучей в пространстве. Доказать корректность полученных распределений случайных величин.

Задание

1. Сформировать равномерное распределение случайных точек (P1, P2, …) внутри треугольника. Треугольник задан тремя координатами в пространстве (V1, V2, V3). Число точек используемых в выборке 100000, число точек, сформированных внутри треугольника также 100000. Доказать, что полученное распределение равномерное и все точки лежат внутри треугольника.

2. Сформировать равномерное распределение случайных точек (P1, P2, …) внутри круга. Круг задан ориентацией нормали N, радиусом круга Rc, и его центром C. Число точек используемых в выборке 100000, число точек, сформированных внутри круга также 100000. Доказать, что полученное распределение равномерное и все точки лежат внутри круга.

3. Сформировать равномерное распределение случайных направлений в пространстве (точек на единичной сфере P1, P2, …). Число точек используемых в выборке 100000, число полученных направлений также 100000. Доказать, что полученное распределение равномерное и для всех выборок были сформированы направления.

4. Сформировать распределение случайных направлений с косинусным распределением плотности вероятности относительно вектора N (единичных векторов, ориентированных по направлению P1 – C, P2 – C, … , плотность вероятности пропорциональна длине вектора P – C). Распределение симметрично относительно вектора N. Число точек используемых в выборке 100000, число полученных направлений также 100000. Доказать, что полученное распределение косинусное и для всех выборок были сформированы направления.

Ход выполнения работы

1. Равномерное распределение в треугольнике

```
generateRandomPoint :: Shape -> IO Point
generateRandomPoint (Triangle p0 vr ur) = do
    xiv <- randomRIO (0 :: Double, 1)
    xiu <- randomRIO (0 :: Double, 1)
    let v = vr - p0
    let u = ur - p0
```

```
    return $
        if xiv + xiu <= 1
            then p0 + v `mul` xiv + u `mul` xiu
            else p0 + v `mul` (1 - xiv) + u `mul` (1 - xiu)
```

1.1 Описание алгоритма

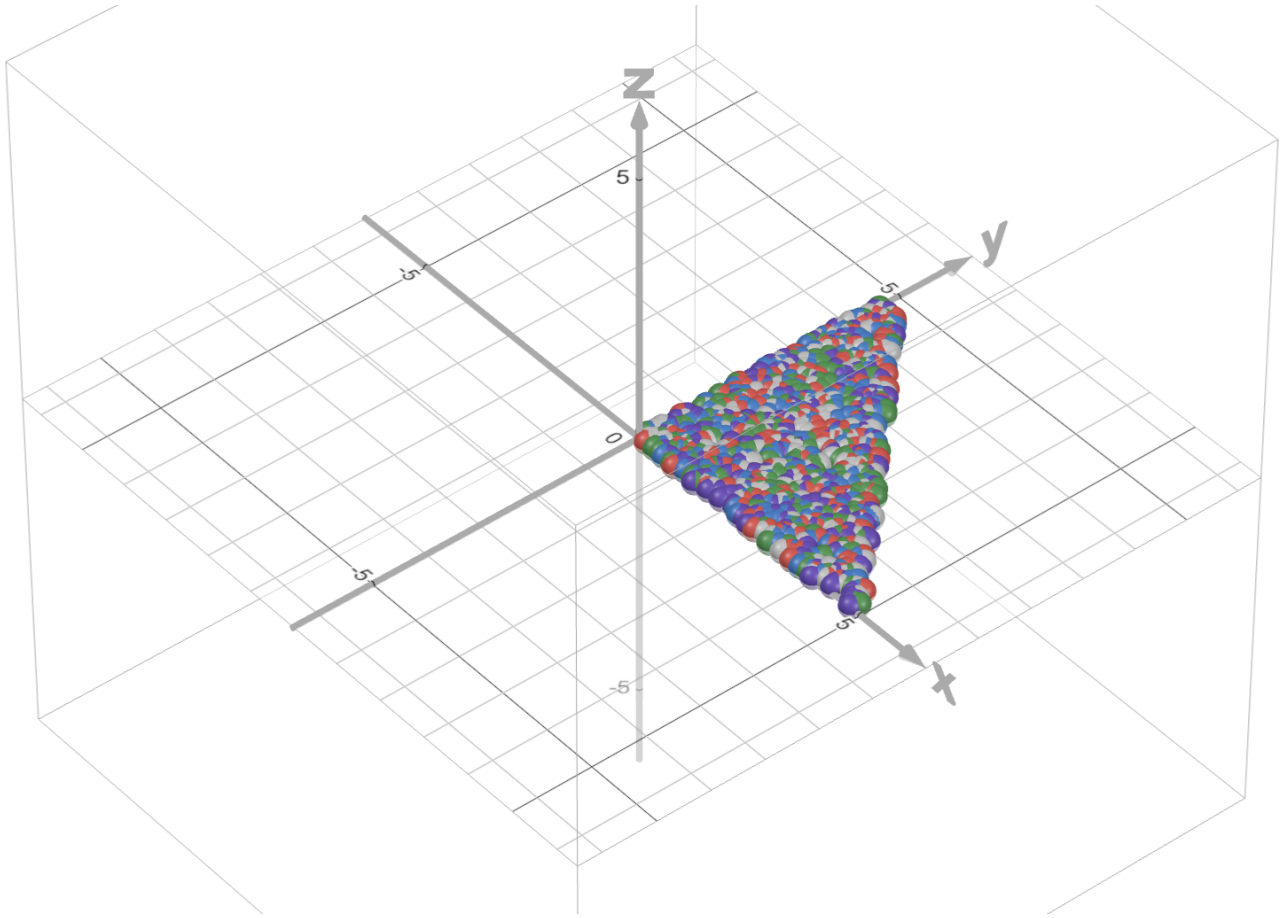
1. Берем ξ_v и ξ_u - равномерно распределенные величины на отрезке $[0; 1]$
2. Формируем векторы $\vec{v}$ и $\vec{u}$ вдоль сторон треугольника от одной точки треугольника
3. Если $\xi_v + \xi_u > 1$, то нужно взять зеркальные координаты, иначе точка будет лежать вне треугольника
4. Прибавляем к точке треугольника смещение по $\vec{v}$ и $\vec{u}$

1.2 Доказательство

```
prop_uniformDistributionTriangle :: Point -> Point -> Point -> Property
prop_uniformDistributionTriangle p1 p2 p3 = monadicIO $ do
    let t = Triangle p1 p2 p3
    points <- run $ mapM (\_ -> generateRandomPoint t) [1 :: Int ..
100000]
    let expectedRatio = 100000 / area t
    assert $
        all
            ( \x ->
                let
                    a = (+) p3 $ flip mul x $ p1 - p3
                    b = (+) p3 $ flip mul x $ p2 - p3
                    t' = Triangle a b p3
                    numInArea = fromIntegral . length . filter (contains
t') $ points
                 in
                    (>=) (0.1 * expectedRatio) $ abs $ expectedRatio -
numInArea / area t'
            )
            [0.1, 0.2 .. 1]
```

Формируется случайный треугольник и разбивается на 10 подобных треугольников. Отношение величины выборки и площади треугольника сформированного треугольника берется за эталон и для каждого подобного треугольника берется такое же отношение. Если полученное отношение отличется от эталонного больше, чем на 10%, то тест провалится

1.3 Визуализация



Визуализация сгенерированных точек для треугольника

```
    let p1 = Point{x = 0, y = 0, z = 0}
    let p2 = Point{x = 5, y = 0, z = 0}
    let p3 = Point{x = 0, y = 5, z = 0}
    let t = Triangle p1 p2 p3
```

2. Равномерное распределение в круге

```
generateRandomPoint :: Shape -> IO Point
generateRandomPoint (Circle n p0 r) = do
    xiv <- randomRIO (0 :: Double, 1)
    xiu <- randomRIO (0 :: Double, 1)
    let v =
            if x n == 0 && y n == 0
                then Point 1 0 0
                else normalize $ Point (negate $ y n / sqrt (y n ** 2 + x
n ** 2)) (x n / sqrt (y n ** 2 + x n ** 2)) 0
    let u = normalize $ n * v
    if (xiv - 0.5) ** 2 + (xiu - 0.5) ** 2 <= 0.25
        then return $ p0 - v `mul` r - u `mul` r + v `mul` (2 * r * xiv) +
u `mul` (2 * r * xiu)
        else generateRandomPoint (Circle n p0 r)
```

2.1 Описание алгоритма

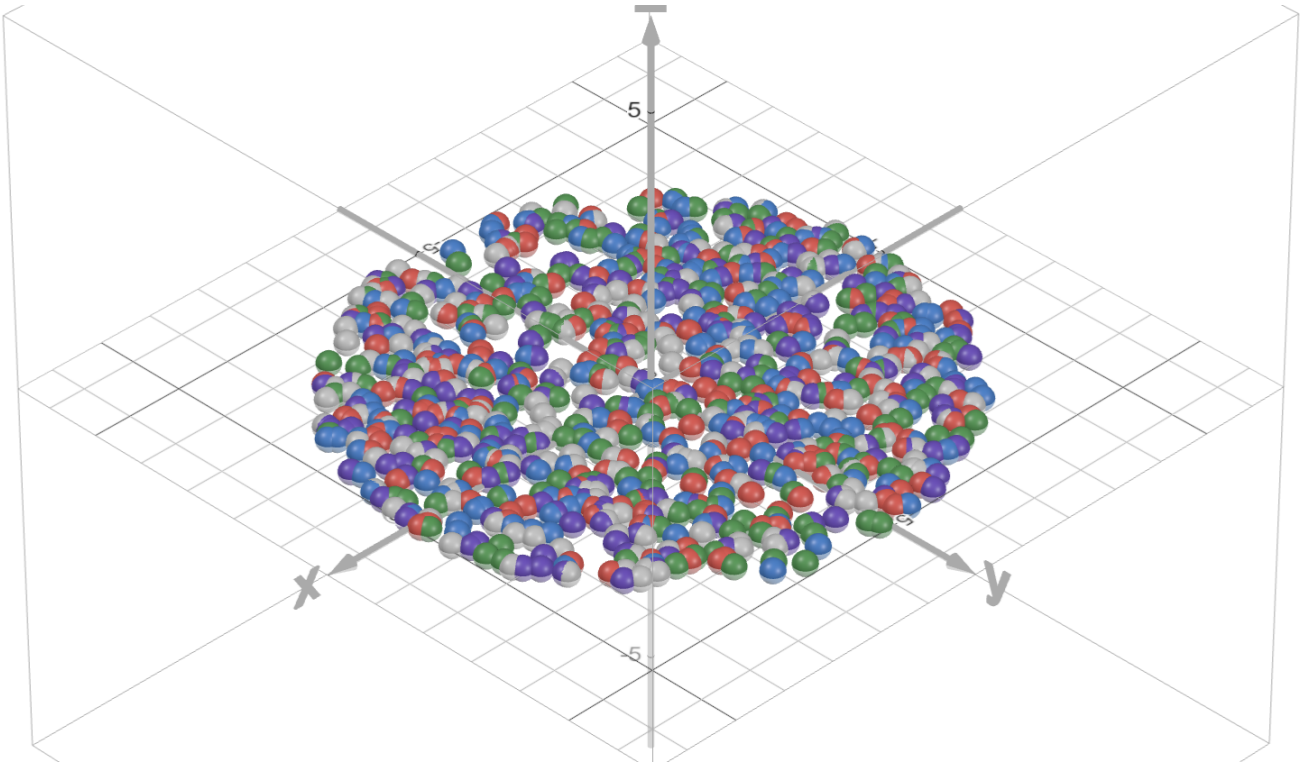
1. Берем ξ_v и ξ_u - равномерно распределенные величины на отрезке $[0; 1]$
2. Формируем систему координат из векторов $\vec{v}$, $\vec{u}$ и $\vec{n}$. Круг будет лежать в плоскости векторов $\vec{v}$ и $\vec{u}$
3. Если $\xi_v и \xi_u$ не попали в единичный круг, то перезапускаем алгоритм
4. От угла квадрата, в который вписан круг, смещаемся по $\vec{v}$ и $\vec{u}$

2.2 Доказательство

```
prop_uniformDistributionCircle :: Point -> Point -> Property
prop_uniformDistributionCircle n p0 = monadicIO $ do
    r <- run $ randomRIO (0.1, 100)
    let c = Circle n p0 r
    points <- run $ mapM (\_ -> generateRandomPoint c) [1 :: Int ..
100000]
    let expectedRatio = 100000 / area c
    assert $
        all
            ( \x ->
                let
                    c' = Circle n p0 $ x * r
                    numInArea = fromIntegral . length . filter (contains
c') $ points
                 in
                    (>=) (0.15 * expectedRatio) $ abs $ expectedRatio -
numInArea / area c'
            )
            [0.1, 0.2 .. 1]
```

Способ доказательства аналогичен генерации точек в треугольнике. Мы формируем 10 кругов с меньшим радиусом и проверяем отклонение от отношения количества точек в области и площади круга

2.3 Визуализация



Визуализация сгенерированных точек для круга

```
    let n = Point{x = 0, y = 0, z = 1}
    let p0 = Point{x = 0, y = 0, z = 0}
    let r = 5
    let c = Circle n p0 r
```

3. Равномерное распределение на поверхности сферы

```
generateRandomVector :: IO Point
generateRandomVector = do
    xih <- randomRIO (0 :: Double, 1)
    xiphi <- randomRIO (0 :: Double, 1)
    let phi = 2 * pi * xiphi
    let h = 2 * xih - 1
    return $ Point ((*) (cos phi) $ sqrt $ 1 - h ** 2) ((*) (sin phi) $
sqrt $ 1 - h ** 2) h
```

3.1 Описание алгоритма

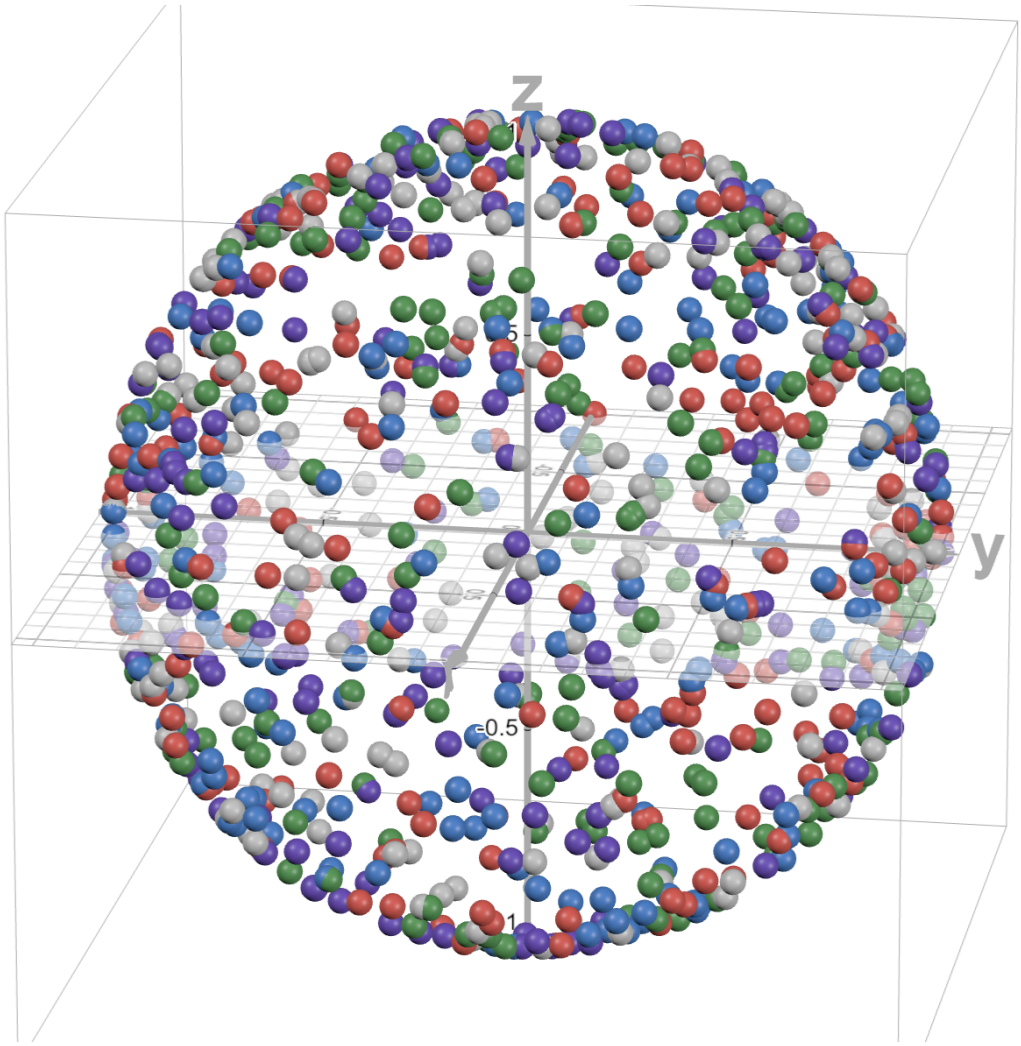
1. Берем ξ_h и ξ_ϕ - равномерно распределенные величины на отрезке $[0; 1]$
2. Формируем случайный угол и высоту на оси сферы
3. Переходим в систему координат XYZ

3.2 Доказательство

```haskell
prop_uniformDistributionSphere :: Property
prop_uniformDistributionSphere = monadicIO $ do
    vecs <- run $ mapM (const generateRandomVector) [1 :: Int .. 100000]
    let expectedRatio = (/) 100000 $ 4 * pi :: Double
    assert $
        all
            ( \h ->
                let
                    numOnStripe = fromIntegral . length $ filter (\v -> z
v <= h && z v > h - 0.1) vecs
                 in
                    (>=) (0.1 * expectedRatio) $ abs $ expectedRatio -
numOnStripe / (2 * pi * 0.1)
            )
            [0.1, 0.2 .. 1]
```

Способ доказательства аналогичен генерации точек в треугольнике. Мы формируем 10 полосок на сфере и проверяем отклонение от отношения количества точек на сфере и площади поверхности сферы

3.3 Визуализация



Визуализация сгенерированных точек для сферы

4. Косинусное распределение на поверхности сферы

```
generateRandomVectorCosDistribution :: Point -> IO Point
generateRandomVectorCosDistribution h = do
    p <- generateRandomVector
    return $ normalize $ h + p
```

4.1 Описание алгоритма

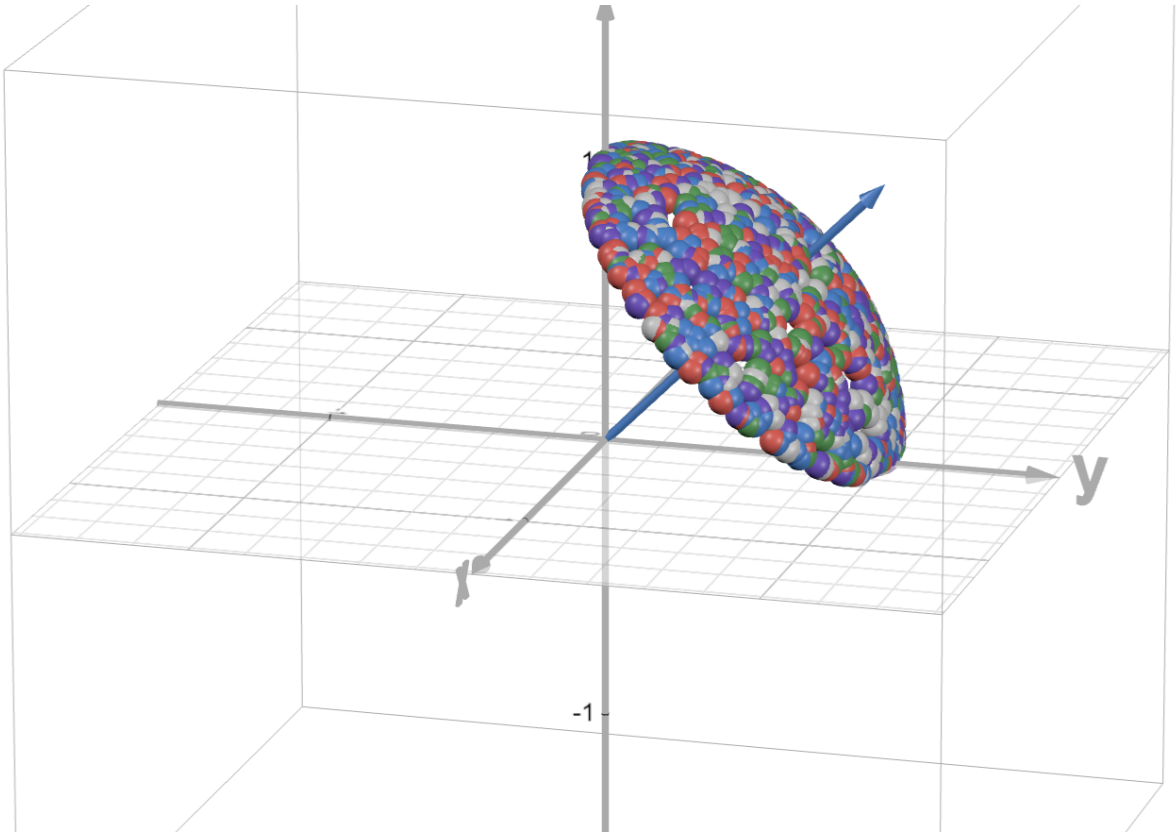
1. Генерируется случайное направление (точка на сфере)
2. Вектор складывается с направлением излучения
3. Полученный вектор нормализуется

4.2 Доказательство

```
prop_cosineDistributionSphere :: Point -> Property
prop_cosineDistributionSphere h = monadicIO $ do
    let n = normalize h
    vecs <- run $ mapM (const $ generateRandomVectorCosDistribution n) [1
:: Int .. 100000]
    let coses = map (dotp n) vecs
    assert $
        all
            ( \k ->
                let
                    numOnStripe = fromIntegral . length $ filter (\c -> c
<= k && c > k - 0.1) coses
                 in
                    (>=) 1e-2 $ abs $ (-) (k ** 2 - (k - 0.1) ** 2) $
numOnStripe / 100000
            )
            [0.1, 0.2 .. 1]
```

Генерируем случайные точки и считаем косинус для каждой сгенерированной точки с вектором $\vec{h}$. Разбиваем все косинусы на промежетки и считаем отношение точек в промежутке ко всем точкам и сравниваем с математическим ожиданием попадания в полоску

4.3 Визуализация



Визуализация сгенерированных точек для сферы

```
let h = Point 0 1 1
```

Вывод

В ходе выполнения лабораторной работы, я познаеомился со способами генерации равномерно распределенных точек на плоских фигурах, а также случайных направлений (точек на сфере).